

TensorFlow.js：重新訓練垃圾留言偵測模型來處理特殊情況

來源：<https://codelabs.developers.google.com/tensorflow-retraining-comment-spam-detection?hl=zh-tw>

步驟數：6

在這個程式碼研究室中，您將瞭解如何使用模型製作工具重新訓練垃圾留言模型，以處理預先訓練模型無法處理的邊緣情況，然後將新的模型重新部署到網頁應用程式。

目錄

1. [1. 事前準備](#)
2. [2. 設定程式碼](#)
3. [3. 找出先前解決方案中的極端情況](#)
4. [4. 重新訓練垃圾留言偵測模型](#)
5. [5. 提供新的機器學習模型](#)
6. [6. 恭喜！](#)

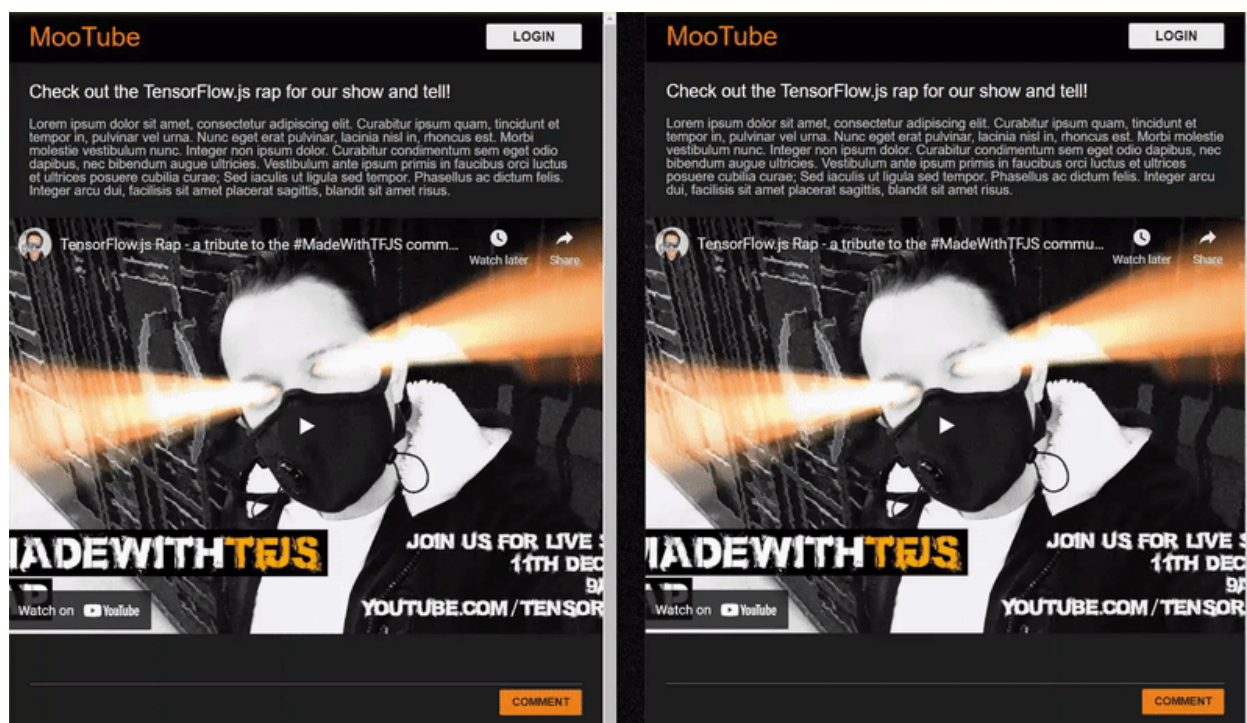
步驟 1 · 約 2 分鐘

1. 事前準備

本程式碼研究室的設計基礎，是本系列中[先前的程式碼研究室的最終結果](#)，也就是使用 TensorFlow.js 偵測垃圾留言。

在[上一個程式碼研究室](#)中，您為虛構的影音網誌建立功能齊全的網頁。您可以在留言傳送至伺服器儲存或傳送至其他連線用戶端前，使用瀏覽器中由 TensorFlow.js 支援的預先訓練留言垃圾訊息偵測模型，篩除垃圾留言。

該程式碼研究室的最終結果如下所示：



雖然這項功能運作良好，但仍有無法偵測到的極端案例。您可以重新訓練模型，以因應模型無法處理的情況。

本程式碼研究室著重於使用自然語言處理 (透過電腦瞭解人類語言的技術)，並說明如何修改您建立的現有網頁應用程式 (強烈建議您[依序完成程式碼研究室](#))，以解決垃圾留言這個非常實際的問題。許多網頁程式開發人員在開發現今數量不斷增加的熱門網頁應用程式時，肯定會遇到這個問題。

在本程式碼研究室中，您將更進一步，根據當前趨勢或熱門討論主題，重新訓練機器學習模型，以因應垃圾訊息內容可能隨時間演變的變化，確保模型與時俱進，並考量這類變化。

必要條件

- 完成本系列的第一個程式碼研究室。
- 具備網頁技術的基本知識，包括 HTML、CSS 和 JavaScript。

建構項目

您將重複使用先前建構的虛構影音網誌網站，其中包含即時留言區，並使用 TensorFlow.js 載入垃圾訊息偵測模型的自訂訓練版本，以便升級網站，在先前會失敗的特殊情況中，獲得更出色的效能。當然，身為網頁程式開發人員和工程師，您可以變更這個假設的使用者體驗，以便在日常工作中處理的任何網站上重複使用，並調整解決方案來配合任何用戶端的使用情境，例如網誌、論壇或某種形式的 CMS (例如 Drupal)。

開始駭客任務吧！

課程內容

您將學會以下內容：

- 找出預先訓練模型無法處理的極端情況
- 重新訓練使用 Model Maker 建立的垃圾內容分類模型。
- 將這個以 Python 為基礎的模型匯出為 TensorFlow.js 格式，以便在瀏覽器中使用。
- 使用新訓練的模型更新代管模型和字典，並檢查結果

本實驗室假設您已熟悉 HTML5、CSS 和 JavaScript。您也會透過「共同研究室」筆記本執行一些 Python 程式碼，重新訓練使用 Model Maker 建立的模型，但您不需熟悉 Python 即可完成這項操作。

步驟 2 · 約 5 分鐘

2. 設定程式碼

您將再次使用 Glitch.com 託管及修改網頁應用程式。如果您尚未完成[必要程式碼研究室](#)，可以在[這裡複製最終結果](#)做為起點。如果您對程式碼的運作方式有疑問，強烈建議您先[完成先前的程式碼研究室](#)，瞭解如何製作這個可運作的網頁應用程式，再繼續進行。

在 Glitch 上，只要按一下「remix this」按鈕，即可將其分支，並建立一組可編輯的新檔案。

替代做法：使用偏好的網路編輯器或在本機作業 如果您想下載程式碼並在本機使用自己的伺服器或不同的線上編輯器作業，只要在您選擇的 Node.js Express 環境中重新建立顯示的檔案，然後將 Glitch 樣板中的程式碼複製並貼到本機環境的每個檔案中即可。

3. 找出先前解決方案中的極端情況

開啟剛複製的已完成網站，並嘗試輸入一些留言，您會發現大部分時間網站都能正常運作，如預期封鎖疑似垃圾內容的留言，並允許合法回覆。

不過，如果你很聰明，嘗試用一些措辭來破解模型，可能就會成功。經過一些嘗試和錯誤，您就能手動建立類似下方的範例。請嘗試將這些內容貼到現有的網頁應用程式中，檢查控制台，並查看留言是否為垃圾內容的回傳機率：

正常發布的留言 (真陰性)：

1. 「哇，我好喜歡這部影片，太棒了。」 垃圾內容機率：47.91854%
2. 「*Totally loved these demos!* 還有其他詳細資料嗎？」 疑似垃圾內容的機率：47.15898%
3. 「如要瞭解詳情，可以前往哪個網站？」 疑似垃圾內容的機率：15.32495%

這很棒，上述所有情況的機率都很低，而且成功通過預設 `SPAM_THRESHOLD` 的 75% 最低機率，然後才採取行動 (在先前程式碼實驗室的 `script.js` 程式碼中定義)。

現在，我們來嘗試撰寫一些更具爭議性的留言，即使不是垃圾內容，也會遭到標記...

將正常留言標示為垃圾內容 (誤判)：

1. 「有人可以提供他戴的口罩網站連結嗎？」 疑似垃圾內容的機率：98.46466%
2. 「我可以在 Spotify 上購買這首歌嗎？請告訴我！」 疑似垃圾內容的機率：94.40953%
3. 「請提供下載 TensorFlow.js 的詳細資訊。」 疑似垃圾內容的機率：83.20084%

糟糕！這些留言應該是正常內容，但系統卻標示為垃圾留言。如何修正？

簡單的做法是提高 `SPAM_THRESHOLD`，讓信賴度超過 98.5%。在這種情況下，系統就會發布這些誤判的留言。瞭解這點後，我們接著來看其他可能的結果...

標示為垃圾內容的留言 (真陽性)：

1. 「這很酷，但請查看我網站上更好的下載連結！」 垃圾內容機率：99.77873%
2. 「我知道有些人可以幫你取得一些藥物，詳情請見我的個人資料」 垃圾內容機率：98.46955%
3. 「查看我的個人資料，下載更多更精彩的影片！<http://example.com>」 垃圾內容機率：96.26383%

好的，這表示原始 75% 的閾值符合預期，但由於您在上一個步驟中將 `SPAM_THRESHOLD` 的信賴度變更為超過 98.5%，這表示這裡會放行 2 個範例，因此閾值可能太高。或許 96% 更好？但如果這麼做，前一節中的其中一則留言 (誤報) 就會被標示為垃圾內容，但該留言的評分是 98.46466%，屬於正當留言。

在這種情況下，建議擷取所有這些真實的垃圾留言，並針對上述失敗案例重新訓練。將門檻設為 96% 後，系統仍會擷取所有真陽性結果，並排除上述 2 個偽陽性結果。只變更一個數字，效果還不錯。

接著繼續...

允許發布的垃圾留言 (誤判為非垃圾內容)：

1. 「前往我的個人資料，下載更多更精彩的影片！」 垃圾內容機率：7.54926%
2. 「參加健身房訓練課程可享折扣，詳情請見個人資料！」 垃圾內容機率：17.49849%
3. 「天啊，GOOG 股票剛剛大漲！把握機會，立即加入！」 垃圾內容機率：20.42894%

對於這類註解，您無法透過進一步變更 `SPAM_THRESHOLD` 值來解決問題。如果將垃圾留言的門檻從 96% 降至約 9%，系統就會將正常留言標示為垃圾留言，其中一則正常留言的評分甚至高達 58%。如要處理這類留言，唯一方法是重新訓練模型，在訓練資料中加入這類極端案例，讓模型學習調整對垃圾內容的看法。

雖然目前只剩下重新訓練模型這個選項，但您也瞭解如何調整決定將某個項目視為垃圾內容的門檻，進而提升效能。以人類的角度來看，75% 似乎很有把握，但對這個模型而言，您需要將信心度提高到接近 81.5%，才能更有效地使用範例輸入內容。

沒有適用於所有模型的魔法值，您必須根據實際資料進行實驗，找出適合各模型的門檻值。

在某些情況下，偽陽性 (或誤判為負面) 可能會造成嚴重後果 (例如醫療產業)，因此您可以將門檻調高，並要求對未達門檻的內容進行更多人工審查。開發人員可以自行選擇，但需要進行一些實驗。

4. 重新訓練垃圾留言偵測模型

在上一節中，您發現模型有許多極端情況會失敗，而唯一的解決方法是重新訓練模型，以因應這些情況。在實際運作的系統中，隨著使用者手動將留言標示為垃圾內容，或管理員審查遭檢舉的留言時發現部分留言並非垃圾內容，並將這些留言標示為重新訓練的資料，您可能會發現這類留言。假設您已收集到大量這類極端情況的新資料 (為獲得最佳結果，您應該盡可能收集這些新句子的一些變化)，接下來我們將說明如何根據這些極端情況重新訓練模型。

預先製作的模特兒回顧

您使用的預先建立模型是由第三方透過 [Model Maker](#) 建立，並使用「平均字詞嵌入」模型運作。

提示：雖然 Model Maker 是專為 TF Lite 使用者 (原生應用程式開發人員) 設計，但它也支援匯出為 TensorFlow.js 模型格式，因此我們也能在網路環境中使用產生的模型，偵測垃圾留言。

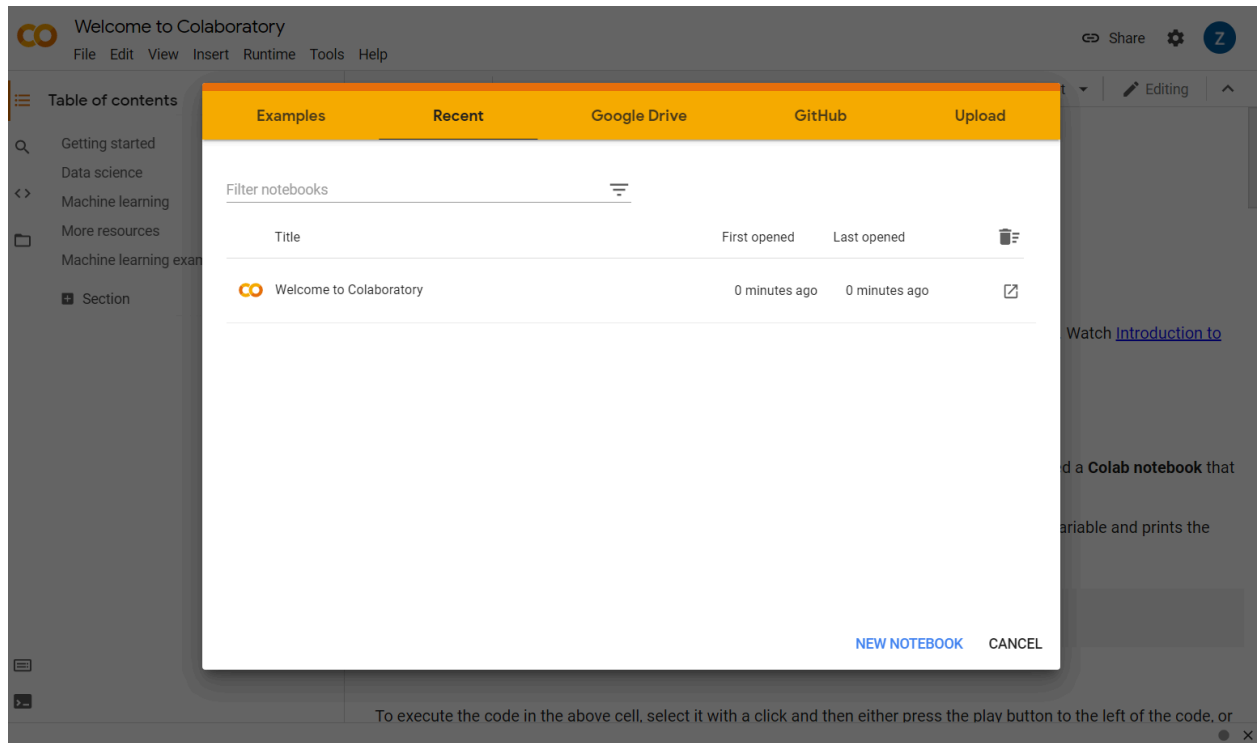
由於模型是使用 Model Maker 建構，您需要暫時切換至 Python 重新訓練模型，然後將建立的模型匯出為 TensorFlow.js 格式，以便在瀏覽器中使用。幸好 Model Maker 讓使用者能輕鬆使用模型，因此您應該能輕鬆跟上我們的腳步。我們會逐步說明整個流程，因此即使您從未用過 Python，也不必擔心！

提示：JavaScript 機器學習工程師通常會使用 TensorFlow.js 完成所有工作，可能透過 Node.js 在伺服器端環境中訓練模型。今天我們將示範如何使用 Model Maker 的程式碼，讓您有信心處理以其他語言 (例如 Python) 建立的模型，並瞭解如何匯出這些模型，以便在 TensorFlow.js 中使用。這項技能非常實用，因為許多研究最初都是由學術界發布，而撰寫時通常會使用 Python。

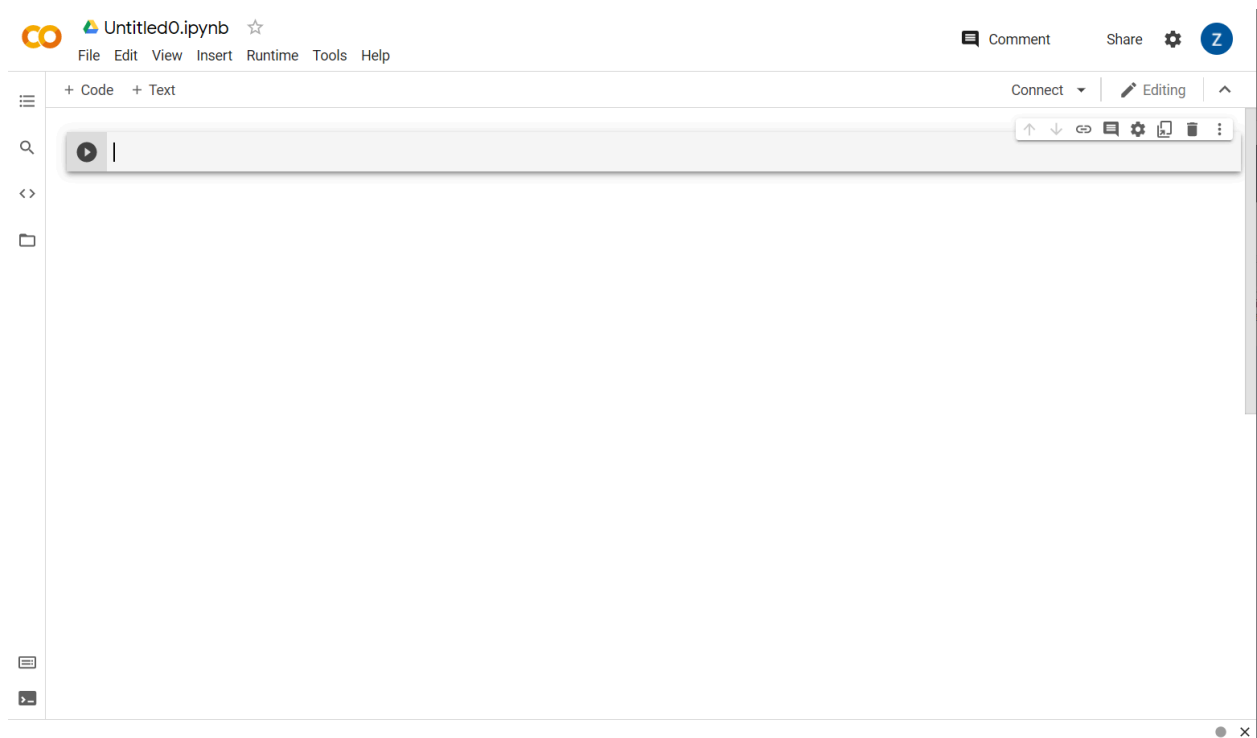
Colab

在本程式碼實驗室中，您不必擔心要設定 Linux 伺服器並安裝各種 Python 公用程式，只要使用「Colab 筆記本」，就能透過網頁瀏覽器執行程式碼。這些筆記本可以連線至「後端」，也就是預先安裝一些項目的伺服器，您可以在網頁瀏覽器中執行任意程式碼，並查看結果。這對快速製作原型或用於這類教學課程非常實用。

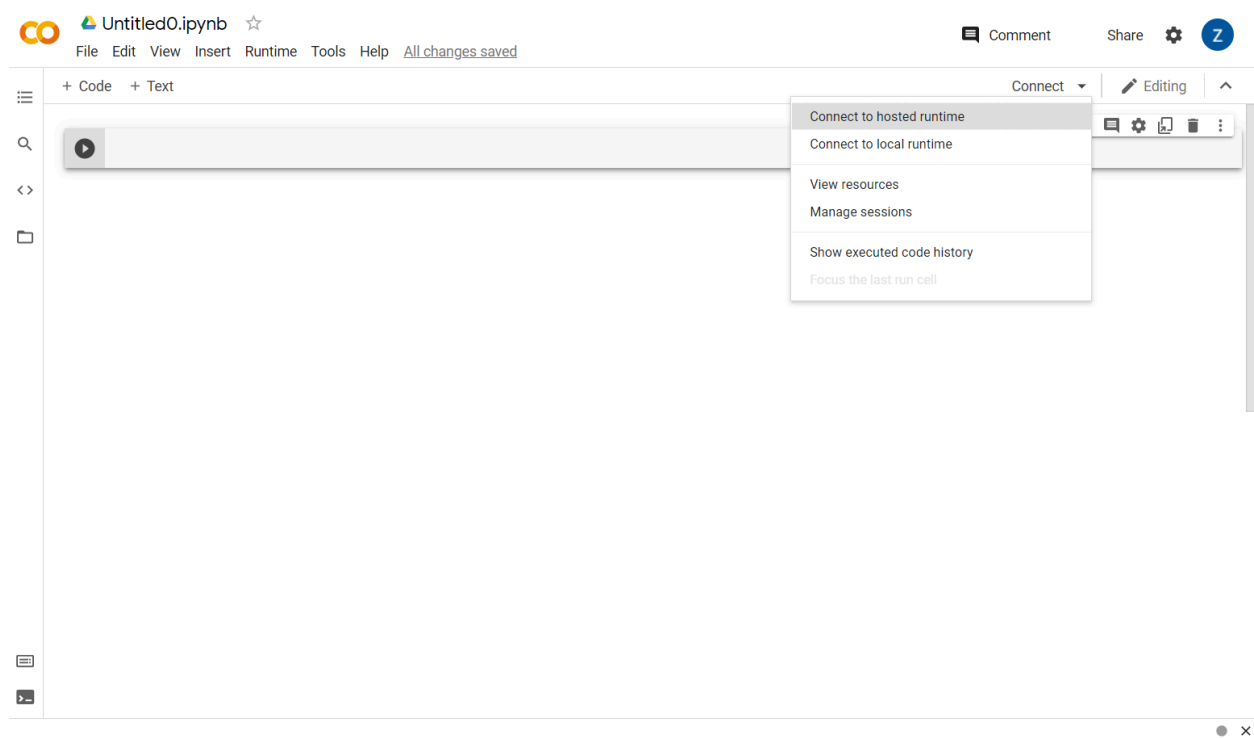
只要前往 colab.research.google.com，就會看到如下所示的歡迎畫面：



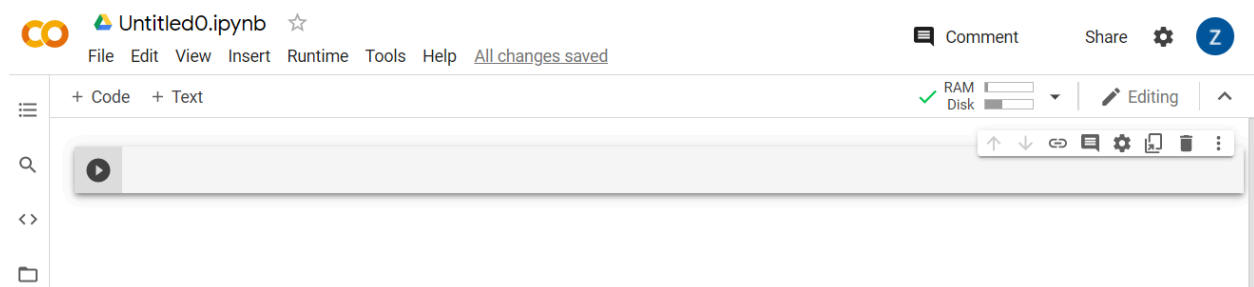
現在按一下彈出式視窗右下方的「New Notebook」按鈕，您應該會看到空白的 Colab，如下所示：



太好了！下一步是將前端 Colab 連線至後端伺服器，以便執行您編寫的 Python 程式碼。方法是按一下右上方的「連結」**Connect**，然後選取「連結至代管的執行階段」**Connect to hosted runtime.**。



連線後，RAM 和磁碟圖示應會顯示在原位置，如下所示：



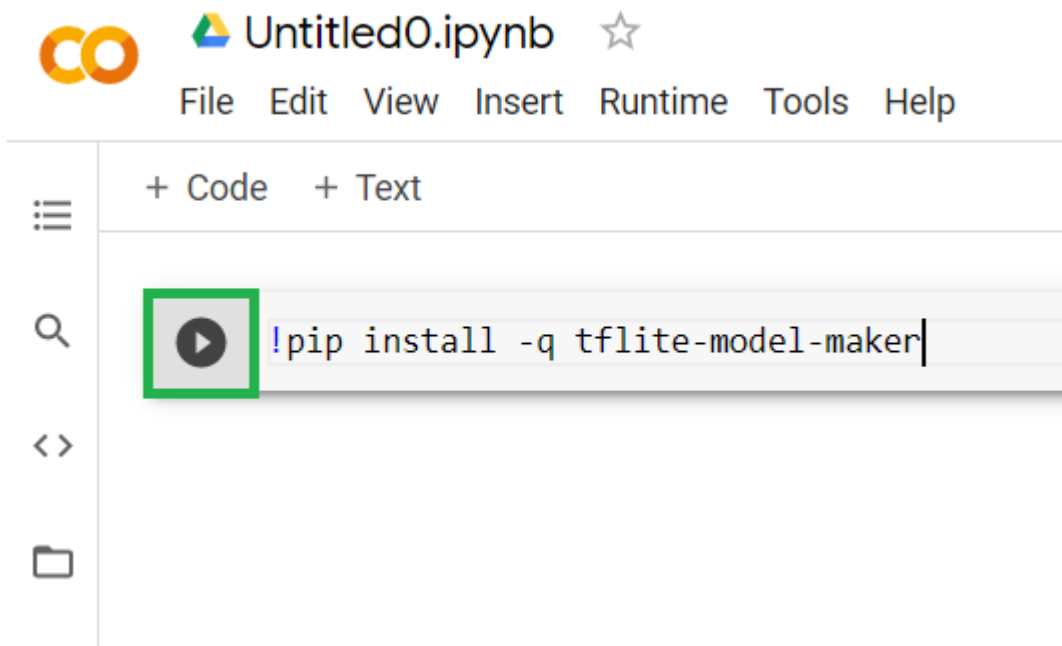
太棒了！現在可以開始使用 Python 重新訓練 Model Maker 模型。只要按照下列步驟進行即可。

步驟 1

在目前空白的第一個儲存格中，複製下列程式碼。系統會使用 Python 的套件管理工具「pip」為您安裝 TensorFlow Lite Model Maker (這與 npm 類似，本程式碼實驗室的大多數讀者可能對 JS 生態系統的 npm 較為熟悉)：

```
!apt-get install libasound-dev portaudio19-dev libportaudio2 libportaudiocpp0
!pip install -q tf-lite-model-maker
```

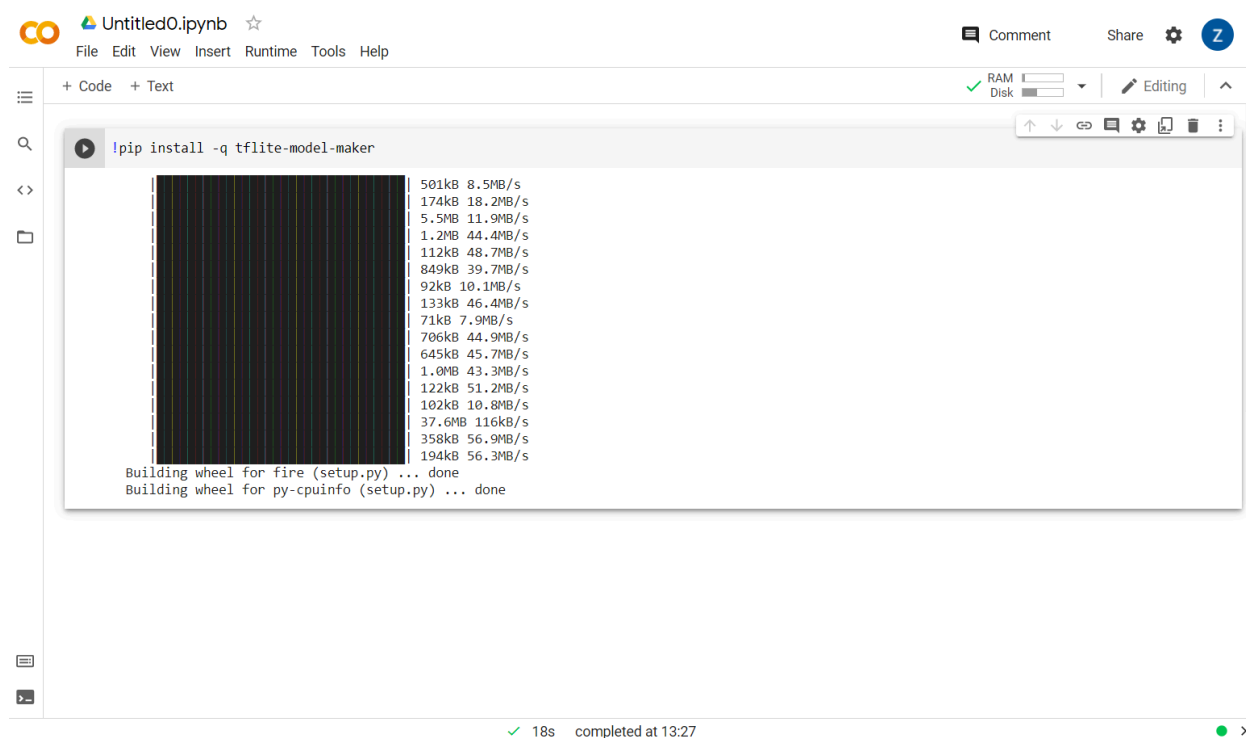
不過，將程式碼貼到儲存格中不會執行程式碼。接著，將滑鼠游標懸停在您貼上上述程式碼的灰色儲存格上，儲存格左側會出現小型「播放」圖示，如下所示：



按一下「播放」按鈕，執行剛在儲存格中輸入的程式碼。

注意：請務必按照本教學課程顯示的順序執行程式碼，否則您稍後在其他儲存格中使用的匯入程式庫或計算變數可能無法使用。

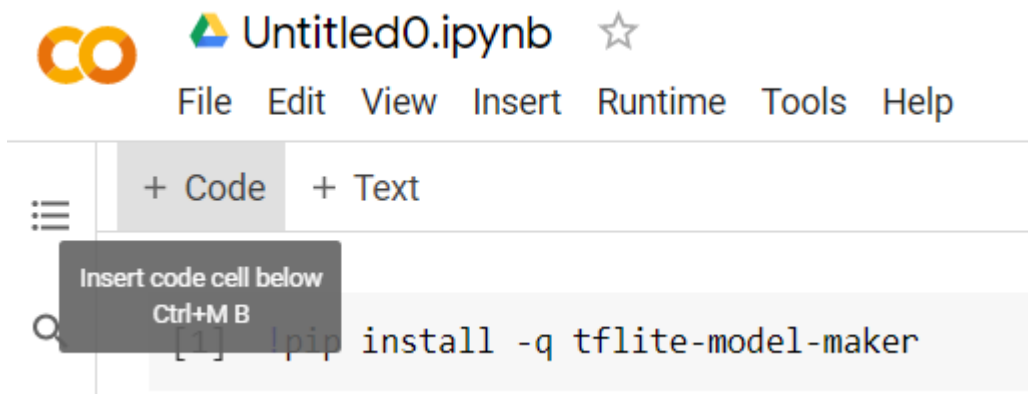
現在您會看到模型製作工具正在安裝：



如圖所示，這個儲存格執行完畢後，請繼續下一個步驟。

步驟 2

接著，如圖所示新增程式碼儲存格，以便在第一個儲存格後貼上更多程式碼，並分別執行：



下一個執行的儲存格會匯入筆記本其餘程式碼需要使用的項目。複製下列內容並貼到新建立的儲存格：

```
import numpy as np
import os

from tfllite_model_maker import configs
from tfllite_model_maker import ExportFormat
from tfllite_model_maker import model_spec
from tfllite_model_maker import text_classifier
from tfllite_model_maker.text_classifier import DataLoader

import tensorflow as tf
assert tf.__version__.startswith('2')
tf.get_logger().setLevel('ERROR')
```

即使不熟悉 Python，這也是相當標準的做法。您只需要匯入一些公用程式，以及垃圾內容分類器所需的 Model Maker 函式。這也會檢查您是否執行 TensorFlow 2.x，這是使用 Model Maker 的必要條件。

最後，就像之前一樣，將游標懸停在儲存格上，然後按下「播放」圖示來執行儲存格，接著新增程式碼儲存格，進行下一個步驟。

步驟 3

接著，請將資料從遠端伺服器下載到裝置，並將 `training_data` 變數設為下載的本機檔案路徑：

```
data_file = tf.keras.utils.get_file(fname='comment-spam-extras.csv',
origin='https://storage.googleapis.com/jmstore/TensorFlowJS/EdX/code/6.5/jm_blog_comments_ext
ras.csv', extract=False)
```

Model Maker 可從簡單的 CSV 檔案 (例如您下載的檔案) 訓練模型。您只需指定哪些資料欄包含文字，哪些資料欄包含標籤。我們會在步驟 5 中說明如何執行這項操作。如要查看檔案內容，請直接下載 CSV 檔案。

眼尖的您會發現這個檔案的名稱是 `jm_blog_comments_extras.csv`，這個檔案只是我們用來產生第一個垃圾留言模型的原始訓練資料 *combined*，加上您發現的新邊緣情況資料，因此所有資料都在一個檔案中。除了要學習的新句子，您也需要用來訓練模型的原始訓練資料。

選用：下載這個 [CSV 檔案](#) 並檢查最後幾行，您會看到先前無法正常運作的極端情況範例。這些資料剛新增至預先建構模型用來訓練的現有訓練資料結尾。

執行這個儲存格，完成後新增儲存格，然後前往步驟 4。

步驟 4

使用 Model Maker 時，您不需要從頭開始建構模型，您通常會使用現有模型，然後根據需求自訂。

Model Maker 提供多種預先學習的模型嵌入，但最簡單快速的入門方式是使用 `average_word_vec`，也就是您在先前的程式碼研究室中用來建構網站的模型。程式碼如下：

```
spec = model_spec.get('average_word_vec')
spec.num_words = 2000
spec.seq_len = 20
spec.wordvec_dim = 7
```

貼到新儲存格後，請執行該儲存格。

瞭解

`num_words`

參數

這是您希望模型使用的字數。你可能會認為越多越好，但一般來說，根據每個字詞的使用頻率，會有一個最佳值。如果您使用整個語料庫中的每個字詞，模型可能會嘗試學習並平衡只使用一次的字詞權重，這並非很有用。在任何文字語料庫中，許多字詞只會使用一到兩次，通常不值得在模型中使用，因為這些字詞對整體情緒的影響微乎其微。因此，您可以使用 `num_words` 參數，根據所需字數調整模型。如果這裡的數字較小，模型就會較小且較快，但由於可辨識的字詞較少，準確度可能會降低。這裡的數字越大，模型就越大，速度也可能越慢。找出最佳平衡點是關鍵，這取決於您身為機器學習工程師，如何找出最適合您用途的設定。

瞭解

`wordvec_dim`

參數

`wordvec_dim` 參數是指您要用於每個字詞向量的維度數量。這些維度基本上是機器學習演算法在訓練時建立的不同特徵，可用來評估任何給定字詞，程式會使用這些維度，盡可能找出在某方面有意義的相似字詞。

舉例來說，如果您有字詞「醫療」程度的維度，則「藥丸」這類字詞在這個維度中可能會獲得高分，並與「X 光」等其他高分字詞建立關聯，但「貓」在這個維度中會獲得低分。如果「醫療維度」與其他可能使用的重要維度結合，或許就能判斷是否為垃圾內容。

如果字詞在「醫療維度」中獲得高分，系統可能會認為將字詞與人體相關聯的第 2 個維度很有用。「腿」、「手臂」、「頸部」等字詞在這裡的分數可能很高，在醫療層面也相當高。

模型可使用這些維度偵測與垃圾內容較有關聯的字詞。或許垃圾郵件比較可能同時包含醫學和人體部位的字詞。

根據研究結果，這個參數的合適值是字數的四次方根。因此，如果我使用 2000 字，建議從 7 個維度開始。如果變更使用的字數，也可以變更這項設定。

瞭解

`seq_len`

參數

模型通常對輸入值非常嚴格。以語言模型來說，這表示語言模型可以分類特定靜態長度的句子。這取決於 `seq_len` 參數，代表「序列長度」。將字詞轉換為數字 (或權杖) 後，句子就會變成一連串的權杖。因此，模型會接受訓練 (在本例中)，分類及辨識含有 20 個符記的句子。如果句子長度超過此限制，系統會截斷句子。如果較短，系統會填補空白，就像本系列[第一個程式碼研究室](#)一樣。

步驟 5 - 載入訓練資料

您先前已下載 CSV 檔案。現在，請使用資料載入器，將這項資料轉換為模型可辨識的訓練資料。

```
data = DataLoader.from_csv(  
    filename=data_file,  
    text_column='commenttext',  
    label_column='spam',  
    model_spec=spec,  
    delimiter=',',  
    shuffle=True,  
    is_training=True)  
  
train_data, test_data = data.split(0.9)
```

如果您在編輯器中開啟 CSV 檔案，會發現每一行只有兩個值，而檔案第一行則以文字說明這些值。通常每個項目都會視為「欄」。您會看到第一欄的描述元是 `commenttext`，且每行的第一個項目都是留言文字。

同樣地，第二欄的描述元是 `spam`，而每行第二個項目會顯示 **TRUE** 或 **FALSE**，表示該文字是否視為垃圾留言。其他屬性會設定您在步驟 4 中建立的模型規格，以及分隔符號字元 (在本例中為半形逗號，因為檔案是以半形逗號分隔)。您也可以設定隨機排序參數，隨機重新排列訓練資料，讓可能相似或一起收集的項目隨機分散在整個資料集中。

接著，您會使用 `data.split()` 將資料分割為訓練和測試資料。`.9` 表示資料集有 90% 會用於訓練，其餘則用於測試。

冷知識：訓練機器學習模型時，通常會保留 10% 到 20% 的原始資料，在模型完全訓練完畢後進行測試。這樣一來，我們就能以從未見過的資料測試產生的模型。這樣我們就能測試其效率，瞭解實際運作情況。如果您使用所有資料進行訓練，可能會過度配適，只考量到這些資料，而無法建立可泛化的模型，以應付未曾見過的情況。

步驟 6 - 建構模型

新增另一個儲存格，我們將在其中新增程式碼來建構模型：

```
model = text_classifier.create(train_data, model_spec=spec, epochs=50)
```

這會使用 Model Maker 建立文字分類器模型，並指定要使用的訓練資料 (已在步驟 4 中定義)、模型規格 (也在步驟 4 中設定)，以及訓練週期數 (本例為 50)。

機器學習的基本原則是模式比對。一開始，系統會載入字詞的預先訓練權重，並嘗試將字詞分組，預測哪些字詞組合在一起時表示垃圾內容，哪些則不是。第一次時，模型才剛開始運作，因此很可能接近 50:50，如下所示：

```
Epoch 1/50  
28/28 [=====] - 4s 20ms/step - loss: 0.6793 - accuracy: 0.7130  
Epoch 2/50  
28/28 [=====] - 0s 4ms/step - loss: 0.6385 - accuracy: 0.8527  
Epoch 3/50  
28/28 [=====] - 0s 4ms/step - loss: 0.5983 - accuracy: 0.8874  
Epoch 4/50  
28/28 [=====] - 0s 4ms/step - loss: 0.5430 - accuracy: 0.9079  
Epoch 5/50  
28/28 [=====] - 0s 4ms/step - loss: 0.4931 - accuracy: 0.8955
```

接著，系統會評估結果，並變更模型權重來調整預測，然後再次嘗試。這是紀元。因此，指定 epochs=50 時，系統會執行 50 次「迴圈」，如下所示：

```
Epoch 48/50
28/28 [=====] - 0s 4ms/step - loss: 0.0398 - accuracy: 0.9872
Epoch 49/50
28/28 [=====] - 0s 4ms/step - loss: 0.0561 - accuracy: 0.9828
Epoch 50/50
28/28 [=====] - 0s 5ms/step - loss: 0.0510 - accuracy: 0.9910
```

因此，當您達到第 50 個訓練週期時，模型會回報準確度高出許多。在此案例中，顯示的百分比為 99.1%！

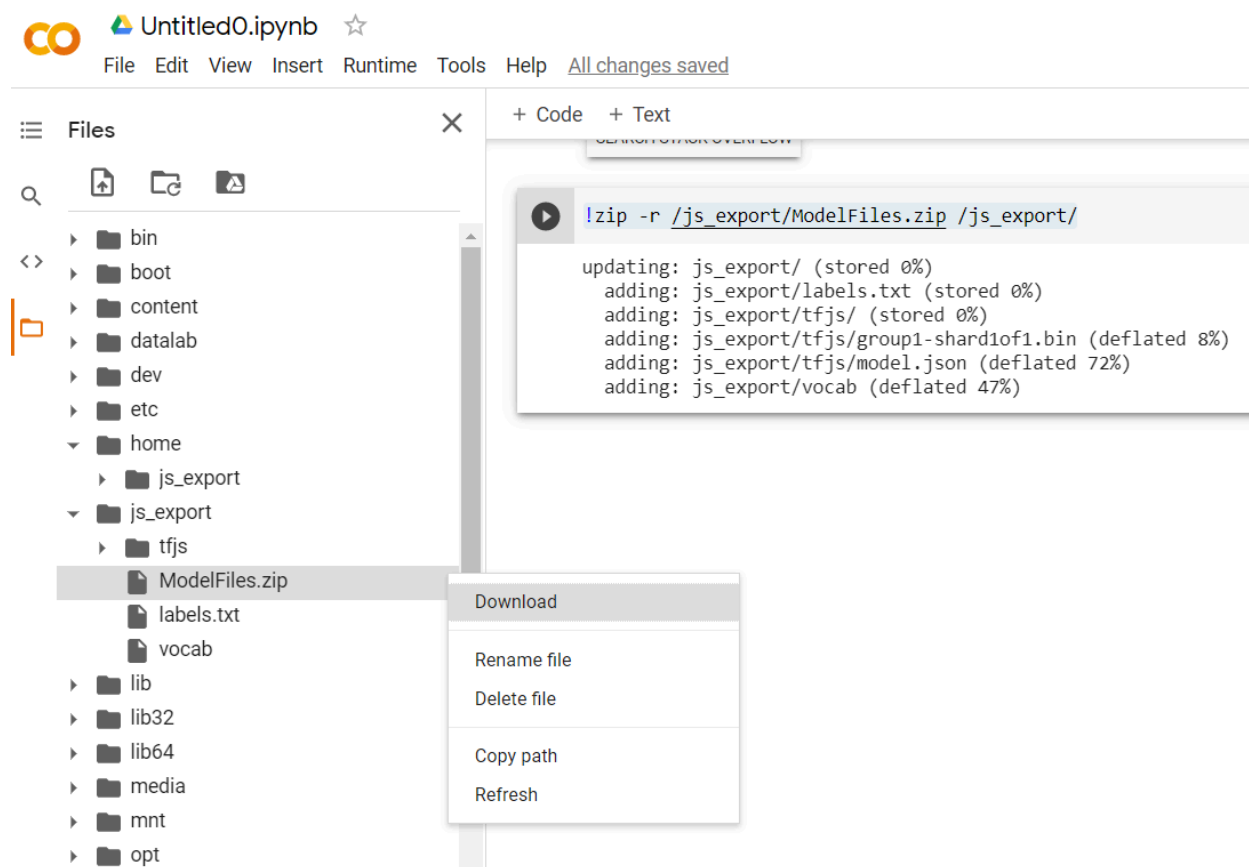
步驟 7 - 匯出模型

訓練完成後，即可匯出模型。TensorFlow 會以自己的格式訓練模型，但必須轉換為 TensorFlow.js 格式，才能在網頁上使用。只要將下列程式碼貼到新的儲存格並執行即可：

```
model.export.export_dir="/js_export/", export_format=[ExportFormat.TFJS, ExportFormat.LABEL,
ExportFormat.VOCAB])
!zip -r /js_export/ModelFiles.zip /js_export/
```

執行這段程式碼後，按一下 Colab 左側的小資料夾圖示，即可前往您在上方匯出的資料夾 (位於根目錄中，您可能需要向上移動一個層級)，並找到 `ModelFiles.zip` 中包含的匯出檔案 ZIP 組合包。

請立即將這個 ZIP 檔案下載到電腦，因為您會像第一個程式碼研究室一樣使用這些檔案：



太好了！Python 部分已完成，現在可以返回您熟悉且喜愛的 JavaScript 領域。呼！

提示：根據預設，您剛編寫的 Colab 筆記本會儲存至 [Google 雲端硬碟資料夾](#) 的「Colab Notebooks」資料夾中。如要使用新的 CSV 檔案重新訓練模型，請重新執行所有程式碼。如果選擇稍後再執行，請務必從頂端開始，依序執行每個儲存格，以免發生任何問題。您也可以執行前編輯任何儲存格。

5. 提供新的機器學習模型

您現在幾乎可以載入模型了。不過，您必須先上傳稍早在程式碼研究室下載的新模型檔案，才能在程式碼中代管及使用該檔案。

首先，如果尚未解壓縮，請解壓縮剛從 Model Maker Colab 筆記本下載的模型檔案。您應該會看到各個資料夾中包含下列檔案：

Name	Date modified	Type	Size
 group1-shard1of1.bin	17/03/2021 14:41	VLC media file (.bin)	56 KB
 labels.txt	17/03/2021 14:41	Text Document	1 KB
 model.json	17/03/2021 14:41	Adobe After Effect...	3 KB
 vocab	17/03/2021 14:41	File	23 KB

這裡有什麼？

- `model.json`：這是構成已訓練 TensorFlow.js 模型的一個檔案。您會在 JS 程式碼中參照這個特定檔案。
- `group1-shard1of1.bin` - 這是二進位檔案，內含匯出 TensorFlow.js 模型的大部分已儲存資料，且必須代管於伺服器上的某個位置，才能在與上述 `model.json` 相同的目錄中下載。
- `vocab` - 這個沒有副檔名的奇怪檔案來自 Model Maker，會顯示如何編碼句子中的字詞，讓模型瞭解如何使用這些字詞。我們將在下一節中深入探討這個主題。
- `labels.txt` - 這只包含模型會預測的結果類別名稱。如果您在文字編輯器中開啟這個模型，會看到「false」和「true」列出，表示「非垃圾內容」或「垃圾內容」的預測輸出。

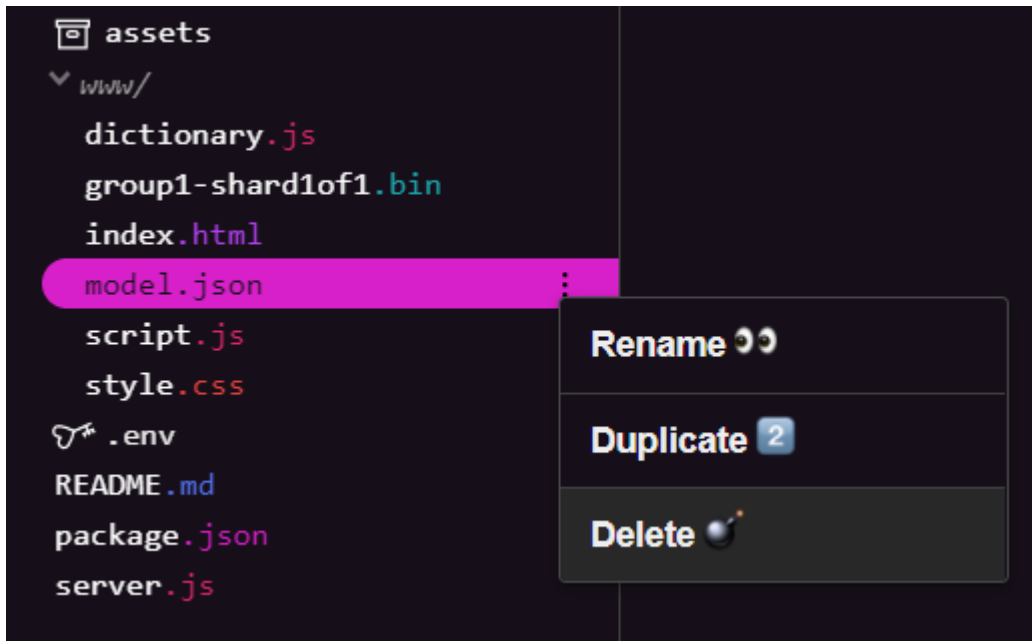
代管 TensorFlow.js 模型檔案

首先，請將產生的 `model.json` 和 `*.bin` 檔案放在網路伺服器上，以便透過網頁存取這些檔案。

注意：如果您未使用 Glitch，而是自行執行 Node.js Express 伺服器，只要將這些檔案**不經修改**放置在 `www` 資料夾中即可提供服務。如果使用 Glitch.com 代管檔案，請按照下列上傳步驟操作。

刪除現有模型檔案

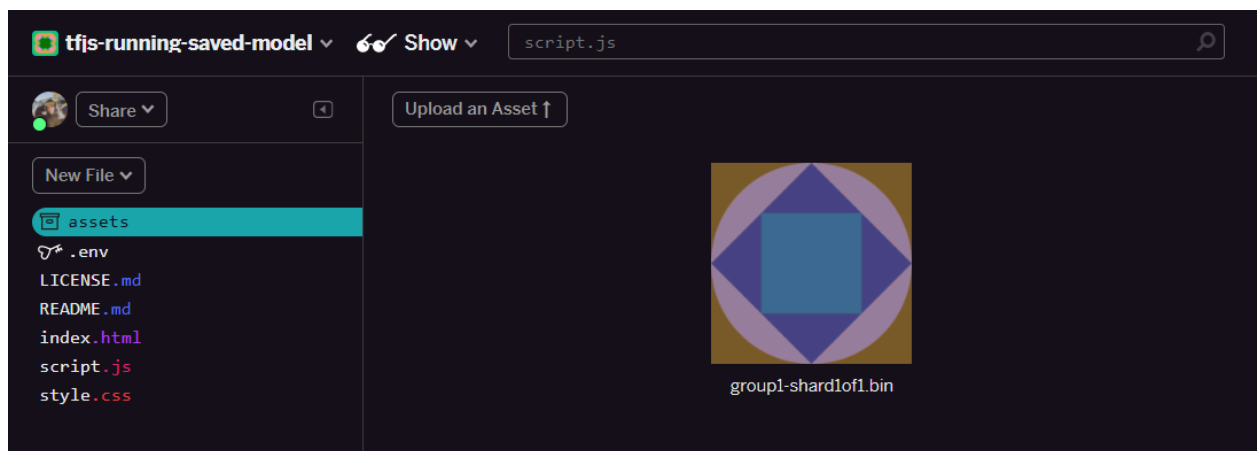
由於您要以本系列第一個程式碼研究室的最終結果為基礎，因此請先刪除已上傳的模型檔案。如果你使用 Glitch.com，只要在左側的檔案面板中查看 `model.json` 和 `group1-shard1of1.bin`，然後點選每個檔案的三點選單下拉式選單，並選取「delete」，如下所示：



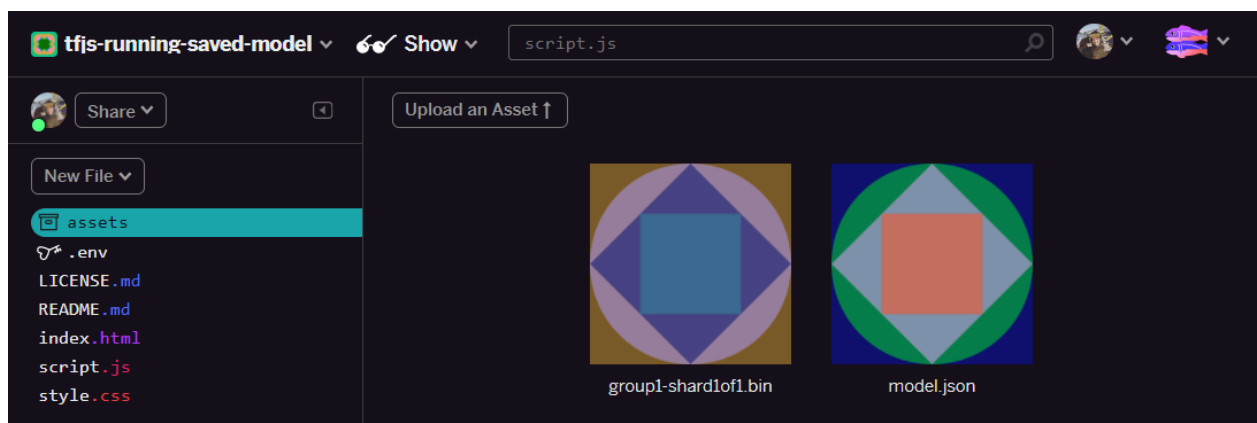
將新檔案上傳至 Glitch

太好了！現在上傳新檔案：

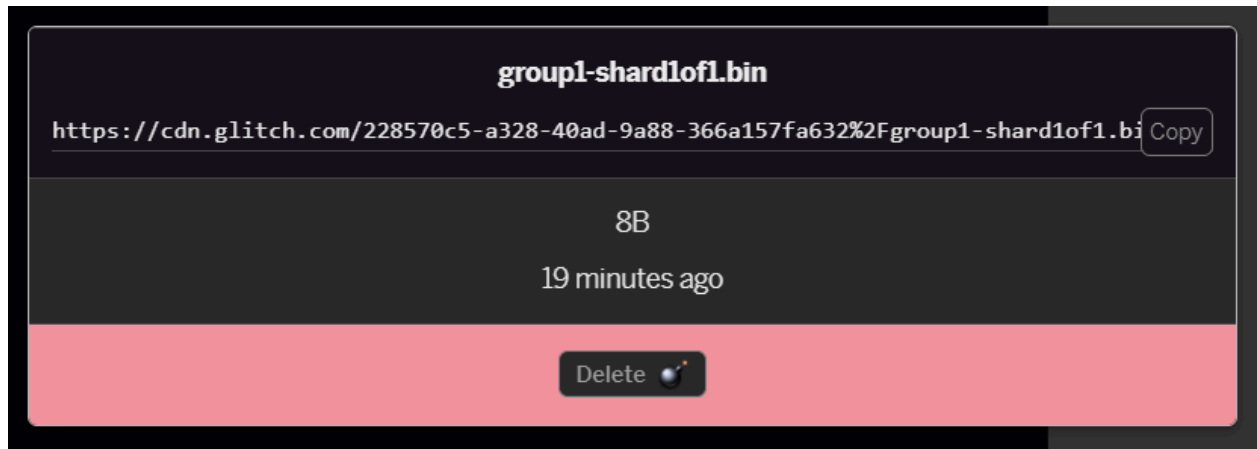
1. 在 Glitch 專案的左側面板中開啟 **assets** 資料夾，然後刪除所有名稱相同的舊素材資源。
2. 按一下「上傳素材資源」，然後選取要上傳至這個資料夾的 `group1-shard1of1.bin`。上傳後應如下所示：



3. 太好了！現在也對 `model.json` 檔案執行相同操作，因此 `assets` 資料夾中應有 2 個檔案，如下所示：



4. 按一下剛上傳的 `group1-shard1of1.bin` 檔案，即可複製該檔案位置的網址。現在請複製這個路徑，如下所示：



5. 現在請點選畫面左下方的「工具」>「終端機」。等待終端機視窗載入完成。
6. 載入後，請輸入下列內容，然後按下 Enter 鍵，將目錄變更為 `www` 資料夾：

terminal:

```
cd www
```

7. 接著，請使用 `wget` 下載剛上傳的 2 個檔案，方法是將下列網址替換為您 Glitch 資源資料夾中為檔案產生的網址（請檢查每個檔案的資源資料夾，找出自訂網址）。

請注意兩個網址之間的空格，以及您需要使用的網址與顯示的網址不同，但看起來相似：

terminal

```
wget https://cdn.glitch.com/1cb82939-a5dd-42a2-9db9-0c42cab7e407%2Fmodel.json?v=1616111344958
wget https://cdn.glitch.com/1cb82939-a5dd-42a2-9db9-0c42cab7e407%2Fgroup1-shard1of1.bin?v=1616017964562
```

太棒了！現在，您已複製上傳至「`www`」資料夾的檔案。

但目前下載的檔案名稱會很奇怪。在終端機中輸入 `ls` 並按下 Enter 鍵，您會看到類似下方的內容：

```
$ ls
1cb82939-a5dd-42a2-9db9-0c42cab7e407%2Fgroup1-shard1of1.bin?v=1616017964562  script.js
1cb82939-a5dd-42a2-9db9-0c42cab7e407%2Fmodel.json?v=1616111344958          index.html  style.css
```

8. 使用 `mv` 指令重新命名檔案。在控制台中輸入下列內容，每輸入一行後按 Enter 鍵：

terminal:

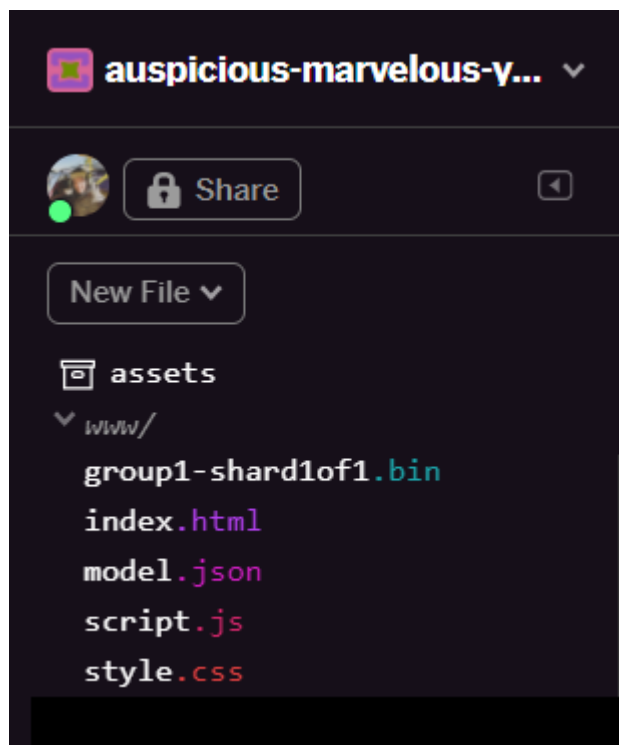
```
mv *group1-shard1of1.bin* group1-shard1of1.bin
mv *model.json* model.json
```

9. 最後，在終端機中輸入 `refresh` 並按下 Enter 鍵，即可重新整理 Glitch 專案：

terminal:

```
refresh
```

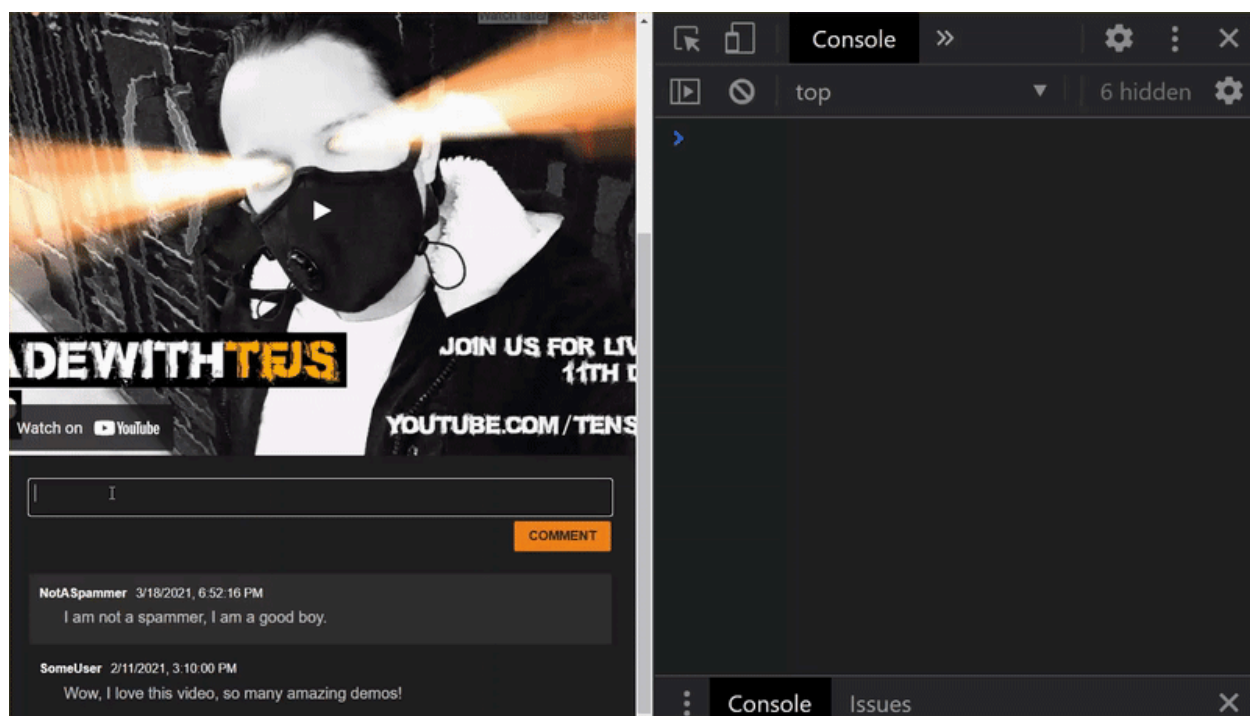
重新整理後，您應該會在使用者介面的 `www` 資料夾中看到 `model.json` 和 `group1-shard1of1.bin`：



太好了！最後一步是更新 `dictionary.js` 檔案。

10. 使用文字編輯器手動將新下載的字彙檔案轉換為正確的 JS 格式，或[使用這項工具](#)，然後將產生的輸出內容儲存為 `www` 資料夾中的 `dictionary.js`。如果您已有 `dictionary.js` 檔案，只要複製並貼上新內容，然後儲存檔案即可。

太厲害了！您已成功更新所有變更的檔案，現在嘗試使用網站時，您會發現重新訓練的模型應能處理發現和學習到的極端情況，如下所示：



如您所見，前 6 封郵件現在已正確分類為非垃圾郵件，而後 6 封郵件則全數標示為垃圾郵件。太完美了！

我們也來試試一些變化，看看是否能順利歸納。原本有失敗的句子，例如：

「omg GOOG stock just shot right up! 把握機會，立即加入！」

現在系統會正確將其分類為垃圾郵件，但如果將其變更為：

「所以 XYZ 股票價值剛才上漲了！請及早購買，以免過期！」

這裡的預測結果為 98% 可能是垃圾內容，即使您稍微變更股票代號和用語，結果仍正確。

當然，如果您真的想破解這個新模型，還是可以做到。這時就需要收集更多訓練資料，盡可能擷取您在網路上可能遇到的常見情況，以獲得最佳結果。在未來的程式碼研究室中，我們會說明如何使用標記的即時資料，持續改善模型。

6. 恭喜！

恭喜！您已成功重新訓練現有的機器學習模型，更新模型以處理發現的極端情況，並使用 TensorFlow.js 將這些變更部署到瀏覽器，以供實際應用。

重點回顧

在本程式碼研究室中，您已完成以下事項：

1. 發現使用預先建立的垃圾留言模型時，無法處理的特殊情況
2. 重新訓練 Model Maker 模型，將您發現的特殊情況納入考量
3. 將新訓練的模型匯出為 TensorFlow.js 格式
4. 更新您的網頁應用程式，使用新檔案

後續步驟

因此這項更新效果極佳，但如同任何網頁應用程式，隨著時間推移，變更在所難免。如果應用程式能隨著時間持續改善，而不是每次都得手動執行這項操作，那就好多了。舉例來說，如果 100 則新留言標示為分類錯誤，您認為可以如何自動執行這些步驟，自動重新訓練模型？只要運用您平常的網頁工程技能，應該就能找出如何建立管道，自動執行這項作業。如果沒有，請別擔心，本系列下一個程式碼研究室將說明如何進行這項操作。

與我們分享你的作品

您也可以輕鬆將今天製作的內容用於其他創意用途，我們鼓勵您發揮創意，持續探索各種可能性。

別忘了在社群媒體上使用 [#MadeWithTFJS](#) 主題標記，你的專案就有機會登上 [TensorFlow 網誌](#)，甚至在 [日後活動](#) 中展示。我們很期待看到你的作品。

更多 TensorFlow.js 程式碼研究室，深入瞭解相關主題

- [使用 Firebase 託管服務，大規模部署及託管 TensorFlow.js 模型。](#)
- [使用預先建立的物件偵測模型和 TensorFlow.js 建立智慧型網路攝影機](#)

值得一訪的網站

- [TensorFlow.js 官方網站](#)
 - [TensorFlow.js 預先建構模型](#)
 - [TensorFlow.js API](#)
 - [TensorFlow.js 展演秀](#) - 激發靈感，看看其他人製作的內容。
-